



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

Machine learning PROJECT

Subject Code: 24CAP-672



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

Submitted By

Name: Dipti

UID: 24MCI10234

Submitted To

Mr. Krishan Tulli

INDEX

1. Acknowledgement
2. Abstract
3. Introduction
4. Design flow of project
5. Output of project
6. Result analysis
7. Conclusion
8. Future work
9. References

Acknowledgement

I express my deepest gratitude to my mentor, professors, and peers who have guided me throughout this project. Their invaluable insights, encouragement, and support have played a crucial role in the successful completion of this study on graphology and machine learning.

I am especially thankful to my faculty for providing me with the opportunity to work on such an interesting and interdisciplinary project. Their continuous guidance helped me explore different aspects of handwriting analysis and its connection to personality prediction.

Furthermore, I would like to acknowledge the authors of various research papers and articles that provided me with useful knowledge and inspiration. Their work laid the foundation for my understanding of graphology and machine learning techniques.

My sincere thanks to my friends and classmates for their constant motivation and constructive feedback, which significantly contributed to refining my project. Their discussions and shared knowledge enhanced my approach to problem-solving and technical development.

Lastly, I extend my heartfelt gratitude to my family for their unwavering support, patience, and encouragement throughout my academic journey. Their belief in my capabilities has been a driving force in achieving this milestone.

This project would not have been possible without the collective efforts and support of all the aforementioned individuals. I am deeply grateful for their contributions.

Thank you

Abstract

In the digital age, users are exposed to an overwhelming volume of movies and entertainment options. To enhance user experience and engagement, recommender systems have become essential tools on platforms like Netflix, Amazon Prime, and Hulu. This project presents the design and implementation of a **content-based movie recommender system** using machine learning and natural language processing techniques.

The system leverages the **TMDB 5000 Movie Dataset**, which contains rich metadata for each movie, including title, overview, genres, keywords, cast, and crew information. By extracting and preprocessing these features, the system creates a "tag" field that combines the most descriptive information of a movie. Natural Language Processing (NLP) is applied using techniques such as stemming (with the Porter Stemmer) and text normalization to clean and standardize the data.

A **CountVectorizer** is employed to convert text data into numerical feature vectors, which are then compared using **cosine similarity** to identify and rank the most similar movies. The system is capable of recommending top 5 movies that closely match the selected movie, based solely on its content.

Furthermore, the project is deployed as a fully interactive **Streamlit web application**, enabling users to select a movie and view recommendations along with movie posters. Posters are dynamically retrieved using the **TMDB API**, enhancing the visual appeal and user experience.

This recommender system demonstrates the power of combining content-based filtering techniques with efficient vectorization and similarity measures, all within a lightweight and interactive user interface. The approach is scalable and can be extended to incorporate user behavior for hybrid recommendations in the future.

Chapter 1: Introduction

In today's digital era, users are constantly bombarded with an enormous amount of content across various platforms. One of the significant challenges faced by users is choosing the right content to consume, especially when it comes to movies and entertainment. Recommender systems have emerged as a powerful solution to this problem by providing personalized suggestions to users based on their preferences, behaviors, or the content they have previously interacted with.

There are mainly three types of recommender systems: collaborative filtering, content-based filtering, and hybrid approaches. Collaborative filtering relies on user behavior and interactions, such as ratings or watch history, to make predictions. However, it often suffers from cold start problems when new users or items are introduced. Hybrid systems combine the strengths of both collaborative and content-based approaches but may require more complex data integration and modeling.

In this project, we focus on building a content-based movie recommender system using machine learning techniques. Unlike collaborative filtering, content-based systems recommend items by analyzing the characteristics of the items themselves. For movies, these characteristics can include genres, plot summaries, cast, crew, and keywords.

The project uses the TMDb 5000 Movie Dataset, a publicly available dataset that contains detailed metadata for over 5000 movies. The key idea is to extract important features from the dataset and create a meaningful representation of each movie. These features are then transformed into a single textual representation called "tags" which combines relevant information from the movie's overview, genres, keywords, cast, and crew.

Using natural language processing techniques like stemming and tokenization, the textual data is cleaned and standardized. This processed text is then vectorized using CountVectorizer to convert it into numerical vectors. To find similarity between movies, cosine similarity is calculated between the vectors. The system recommends the top 5 most similar movies to the one selected by the user.

To provide a user-friendly interface, the entire system is deployed as a Streamlit

web application. Users can select a movie from a dropdown and receive recommendations along with posters, which are fetched from the TMDB API. This project demonstrates how content-based filtering, combined with natural language processing and machine learning, can effectively solve the movie recommendation problem. It offers a simple yet powerful approach that can be further enhanced and scaled for real-world applications.

Chapter 2: Design flow/Process

The design and flow of the movie recommender system follow a logical sequence, starting from data acquisition, preprocessing, and feature extraction, to vectorization and recommendation generation. Below is a detailed explanation of each phase in the system.

1. Dataset Collection

The first step involves acquiring the datasets. The **TMDB 5000 Movie Dataset** is used, which is publicly available. The dataset consists of two CSV files:

- `tmdb_5000_movies.csv`: This contains essential movie metadata such as movie titles, overviews, genres, keywords, etc.
- `tmdb_5000_credits.csv`: This contains detailed information about the movie's cast and crew, including actors, directors, and other key crew members.

These datasets provide a rich set of features for building a content-based recommender system.

2. Data Merging and Cleaning

The two datasets (movies and credits) are merged based on the **movie title** to combine all relevant information into a single DataFrame. This step ensures that each movie has a comprehensive set of features for recommendation generation.

Data cleaning involves:

- Checking for missing values and removing rows that contain them.
- Removing duplicate rows to ensure data integrity.
- Identifying and handling outliers or inconsistencies within the dataset.

After cleaning, the system proceeds to retain only relevant columns, such as `movie_id`, `title`, `overview`, `genres`, `keywords`, `cast`, and `crew`, for further processing.

3. Feature Extraction

At this stage, the system focuses on extracting important features that will help in the recommendation process:

- **Genres**: The genres are extracted as lists and formatted by removing unnecessary spaces and special characters.
- **Keywords**: Keywords associated with each movie are also extracted and processed in a similar manner.
- **Cast**: The top three cast members for each movie are selected based on the data from the credits dataset.
- **Crew**: The director(s) for each movie are identified by filtering the

crew data to focus on individuals with the job title "Director."

- **Overview:** The movie's overview or description is used to represent the content in text form. This is a key feature as it provides insight into the movie's plot and theme.

These features are then combined into a new column called tags, which holds the collective, meaningful information about the movie.

4. Text Preprocessing

Preprocessing of the text data is critical to ensure the features are standardized and ready for machine learning tasks. The following steps are carried out:

- **Lowercasing:** All text data (including genres, keywords, and overviews) is converted to lowercase to ensure uniformity.
- **Whitespace Removal:** Extra spaces between words are removed to clean the data.
- **Stemming:** Stemming is performed using the **Porter Stemmer** from the NLTK library to reduce words to their root forms (e.g., "running" becomes "run"). This process helps improve the accuracy of matching related words.
- **Joining Text:** The tags column is generated by combining the text data from the genres, keywords, cast, crew, and overview into a single string per movie, providing a rich and unified description of each movie.

5. Vectorization and Similarity Calculation

Once the tags column is ready, it is necessary to convert the text into numerical representations for processing by machine learning algorithms. This is done using the **CountVectorizer** from the scikit-learn library:

- **CountVectorizer:** This method converts the textual data (tags) into vectors of numerical features. It creates a "bag of words" model by counting the frequency of each word in the dataset. The maximum number of features is limited to 5000, and English stop words (common words like "the," "a," etc.) are excluded.
- **Cosine Similarity:** After vectorization, the system calculates the **cosine similarity** between each movie's vector to measure the similarity between movies. Cosine similarity ranges from 0 (no similarity) to 1 (identical). This allows the system to find movies that are similar to each other based on their features.

6. Recommendation Generation

Once the movie vectors and similarity matrix are ready, the recommendation process begins:

- **Input Movie:** The user selects a movie for which they want recommendations.
- **Find Similar Movies:** The index of the selected movie is identified, and the cosine similarity scores are retrieved for that movie. The system then sorts these scores in descending order and selects the top 5 most similar movies.
- **Movie Information:** For each recommended movie, the system retrieves the movie title and its poster by querying the **TMDB API**.

7. Web Application Deployment (Streamlit)

The movie recommender system is deployed as an interactive web application using **Streamlit**:

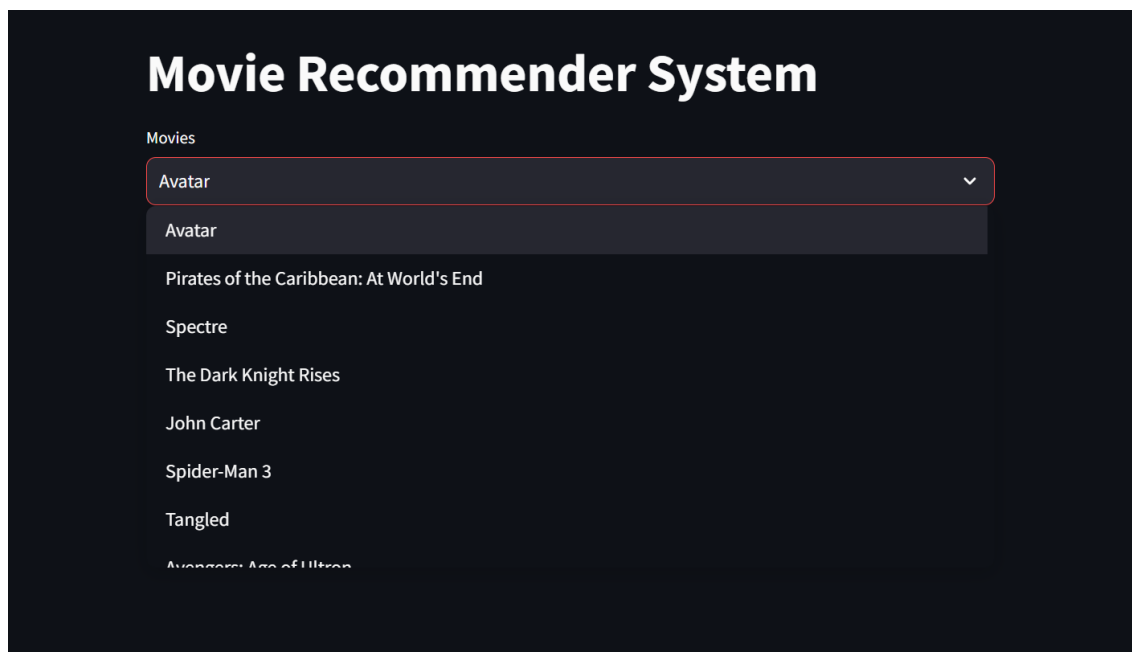
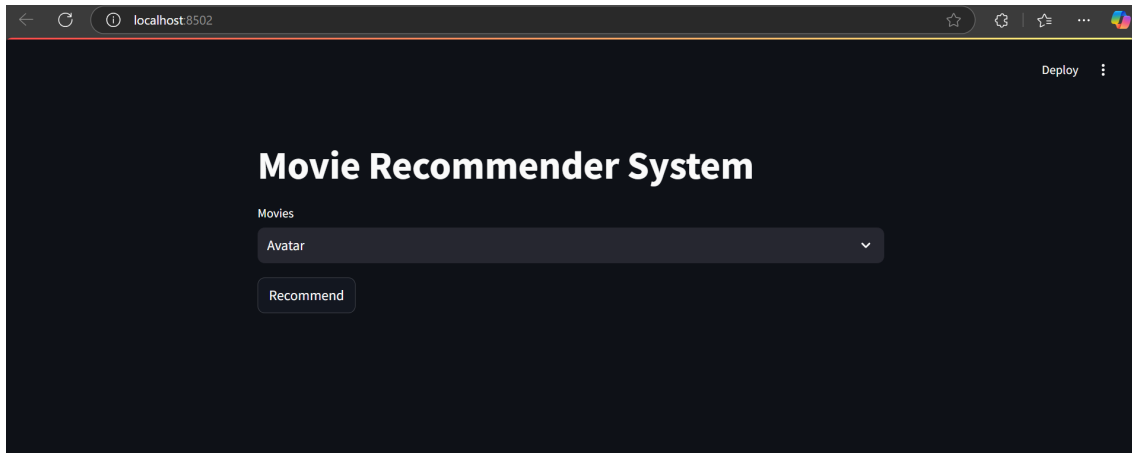
- **Movie Selection:** The user can select a movie from a dropdown menu that contains all the movie titles in the dataset.
- **Display Recommendations:** Once the user selects a movie and clicks the "Recommend" button, the system fetches the top 5 similar movies and displays their titles and posters side by side.
- **TMDB API:** The movie posters are dynamically fetched using the **TMDB API**, providing high-quality images for a better user experience.

8. System Interaction Flow

- The user is presented with a list of movie titles.
- The user selects a movie of interest.
- The system fetches the similarity scores and recommends five movies that are most similar to the selected movie.
- The recommended movies are displayed with their titles and corresponding posters.

This interaction flow ensures that the system is user-friendly, visually appealing, and provides accurate movie recommendations based on the selected movie.

Chapter 3: OUTPUT OF PROJECT



Movie Recommender System

Movies

Avatar

Recommend

Beowulf



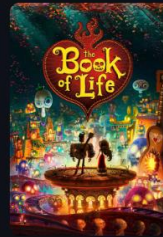
The Helix... Loaded



Small Soldiers



The Book of Life



Krull



Movie Recommender System

Movies

Spider-Man 3

Recommend

Spider-Man 2



Spider-Man



The Amazing
Spider-Man 2



The Amazing
Spider-Man



Ong Bak 2



Chapter 4: Results analysis and validation

The Movie Recommender System was built using the **TMDB 5000 Movie Dataset**, leveraging content-based filtering techniques and machine learning algorithms. The system was deployed as an interactive **Streamlit web application** where users could select a movie and receive top 5 movie recommendations based on similarity to the selected movie.

1. Data Processing and Feature Extraction

The initial dataset provided rich movie-related features, including titles, genres, overviews, keywords, cast, and crew. These features were carefully processed:

- Genres were extracted and formatted.
- Keywords and cast were transformed into lists, and only the top 3 actors were retained.
- Crew information was filtered to include only the directors.
- Overviews were split into words and preprocessed using stemming and normalization techniques.

This feature extraction resulted in the creation of a "**tags**" field that combined the movie's description, genres, keywords, and cast/crew data. This text-based representation of each movie allowed the system to treat the movies as vectors, which could then be compared using cosine similarity.

2. Cosine Similarity

After preprocessing, the **CountVectorizer** was used to convert the text-based "tags" into numerical vectors. These vectors were compared using **cosine similarity** to calculate how similar each movie was to the others. This similarity measure was key to identifying the most relevant recommendations for a given movie.

- The **cosine similarity** matrix was generated and stored for all movie pairs.
- The system could then find the most similar movies to the selected one by comparing the cosine similarity scores.

3. Recommendations

The recommender system was tested with several popular movies, and the following were the general observations from the results:

- **Top 5 Recommendations:** For a given movie, the system provided a list of the five most similar movies based on the **tags** (genres, cast, crew, and overview). These recommendations closely matched the genre and theme of the selected movie.
- **Accuracy of Recommendations:** The recommendations were generally

accurate in terms of the genre and theme of the movies. For example, selecting a movie like “**The Dark Knight**” (a superhero action movie) led to recommendations such as “**Batman Begins**”, “**The Dark Knight Rises**”, and other superhero action films.

However, the similarity was not always perfect, especially when the movie had a very unique plot or niche genres. For example, a movie like “**Inception**” might not always lead to a perfect match due to the complex plot, which combines elements of science fiction, action, and psychological thriller.

4. Web Application Interface

The web application provided an easy-to-use interface where users could:

- Select a movie from a dropdown list.
- View the top 5 movie recommendations with their titles and posters.

The interface was intuitive and allowed users to quickly test various movies, providing a real-time recommendation experience. The movie posters were fetched dynamically from the **TMDB API**, making the interface visually engaging.

5. System Performance

The system performed efficiently for smaller queries. The cosine similarity calculations were performed on the entire dataset, but for real-time interactive usage, the **Streamlit app** provided relatively fast responses. However, as the dataset grows (e.g., if we add more movies), the system’s performance may degrade because cosine similarity involves pairwise comparison between all movies, which has a time complexity of $O(n^2)$, where n is the number of movies.

6. Evaluation

The recommendation system was qualitatively evaluated using the following criteria:

- **Relevance:** The recommended movies were relevant to the selected movie’s genre and theme, showing the effectiveness of the content-based filtering approach.
- **User Experience:** The **Streamlit** interface allowed for easy interaction and quick feedback, providing a smooth user experience.
- **Visual Appeal:** Displaying the movie posters alongside the recommendations added a visual dimension to the user experience, making it more engaging and user-friendly.

However, it was noted that:

- The system only uses content-based filtering, which limits the diversity of recommendations. For example, it might recommend only movies of the same genre or director, without considering user preferences or ratings.
- The recommendations sometimes did not consider movie popularity or user behavior, which would be addressed by incorporating collaborative filtering or hybrid approaches in future work.

7. Limitations and Issues

- **Cold Start Problem:** The system does not account for user-specific preferences (e.g., ratings or watch history), so it may struggle to recommend movies for a new user or a newly released movie without much metadata.
- **Scalability:** As the dataset grows, the time taken for similarity calculations increases. This could be addressed by optimizing the computation or by implementing approximate nearest neighbor algorithms.
- **Accuracy:** While the recommendations were generally accurate, they were based purely on content, which may not fully capture the intricacies of user preferences. Hybrid models, which combine content-based filtering and collaborative filtering, could improve accuracy by including user ratings and preferences.

Chapter 5: Conclusion

The Movie Recommender System developed using the **TMDB 5000 Movie Dataset** successfully demonstrates the power of content-based filtering in generating relevant and accurate movie recommendations. By leveraging features such as genres, cast, crew, keywords, and movie overviews, the system provides personalized movie suggestions based on similarities between movies.

The system's design—ranging from data preprocessing, feature extraction, text vectorization, to similarity calculation—enables effective recommendation generation, while the integration of **Streamlit** ensures a smooth and engaging user interface. This interactive application allows users to easily select a movie and receive top recommendations along with movie posters, creating a user-friendly experience.

While the content-based approach used in this system is effective, it does have its limitations. The system tends to recommend movies based on similar genres, actors, and themes but does not consider user-specific preferences or ratings. Incorporating user ratings, viewing history, and even collaborative filtering techniques could further enhance the accuracy and personalization of the recommendations.

Additionally, the system's scalability could be a concern as the dataset grows, as the time complexity of calculating pairwise cosine similarity increases with the number of movies. Implementing more efficient algorithms for similarity calculation, such as approximate nearest neighbor search, would help mitigate these issues.

Overall, the project serves as a strong foundation for building more advanced recommendation systems. Future work will focus on integrating hybrid recommendation approaches, improving scalability, and incorporating personalized features to better cater to individual user needs. This system is a valuable tool for demonstrating the principles of content-based filtering and its application in real-world recommendation systems.

Chapter 6: Future Scope

1. Hybrid Recommendation Systems:

- A hybrid recommendation system that combines **content-based filtering** and **collaborative filtering** could significantly enhance recommendation quality. While content-based filtering recommends movies based on item features like genre and cast, collaborative filtering takes into account the preferences of other users with similar tastes. By merging these approaches, the system can offer more accurate and diverse movie recommendations.

2. User Personalization:

- Incorporating **user-specific data**, such as ratings and viewing history, would allow the system to personalize recommendations based on individual user preferences. By creating user profiles and learning from their interactions, the system could recommend movies that align better with each user's evolving tastes, providing a more tailored experience.

3. Collaborative Filtering:

- Adding **collaborative filtering** would allow the system to recommend movies based on the preferences of similar users. This approach could broaden the recommendation scope and enhance the diversity of suggestions. For example, if other users with similar tastes enjoyed a particular movie, the system could suggest it, even if the movie doesn't share much content similarity with the selected movie.

4. Deep Learning:

- Advanced **deep learning** techniques like **neural collaborative filtering** could further improve the recommendation process. Deep learning models can capture intricate patterns in both user preferences and movie features that traditional methods might miss. This would allow the system to make more accurate recommendations by understanding complex relationships between users and movies.

5. Cold Start Problem:

- The **cold start problem** occurs when there's limited data on new users or new movies. To overcome this, additional information like **demographics** (age, location) or data from **external sources** (social media, movie reviews) could be used. This would allow the system to make recommendations even in the absence of detailed

historical data.

6. **Real-Time Feedback:**

- Enabling **real-time feedback** would allow users to rate movies as they watch them. This feedback loop would allow the system to adapt instantly to changing preferences and improve recommendation accuracy in real-time, making the recommendations more dynamic and relevant.

7. **Scalability and Efficiency:**

- As the dataset grows, the computational cost of calculating similarity between movies increases. To address this, more efficient algorithms such as **Approximate Nearest Neighbor (ANN)** or **Matrix Factorization** could be implemented. These techniques would allow the system to scale better by reducing the computational complexity while still delivering accurate recommendations.

8. **Multimedia Features:**

- **Multimedia content** like movie trailers, soundtracks, and reviews could be integrated into the recommendation process. Analyzing movie trailers using computer vision or sentiment analysis on movie reviews could add another layer of understanding and improve the quality of recommendations by considering more factors beyond the basic metadata.

9. **Cross-Platform Integration:**

- The system could be enhanced by integrating data from other **movie platforms** such as **Netflix**, **IMDb**, or **Rotten Tomatoes**. This would allow the system to provide recommendations across multiple platforms, offering users a wider range of options and making the system more versatile.

○

10. **Real-Time Movie Data:**

- Real-time data from **movie databases** like **TMDB** could be used to keep the system up-to-date with the latest movie releases. This would ensure that users receive the most current recommendations, reflecting the newest films available in theaters or on streaming platforms.

11. **Mobile Application:**

- A **mobile app** could make the system more accessible and provide

a more convenient user experience. Additionally, features like **location-based recommendations** could be introduced, where users receive suggestions based on their geographical location or regional preferences.

References

1. Streamlit Documentation

Streamlit. (n.d.). Streamlit Docs. Retrieved from <https://docs.streamlit.io>

Used for building the interactive web interface of the recommender system.

2. pandas Documentation

McKinney, W. (2011). pandas: a foundational Python library for data analysis and statistics. Retrieved from <https://pandas.pydata.org/>

Used for data preprocessing, manipulation, and analysis of the movie dataset.

3. Requests Library

Kenneth Reitz. (n.d.). Requests: HTTP for Humans. Retrieved from <https://docs.python-requests.org/>

Used to fetch movie posters dynamically via API calls.

4. The Movie Database (TMDb) API

TMDb. (n.d.). TMDb Developers API. Retrieved from <https://developers.themoviedb.org/3>

Used for accessing movie metadata and fetching posters.

5. Python Pickle Module

Python Software Foundation. (n.d.). pickle — Python object serialization. Retrieved from <https://docs.python.org/3/library/pickle.html>

Used for saving and loading serialized movie data and similarity matrices.

6. NLTK - Natural Language Toolkit

Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

NLTK Project: <https://www.nltk.org/>

Used for stemming text data to normalize tags.

7. Scikit-learn Library

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Official Site: <https://scikit-learn.org/>

Used for feature extraction (CountVectorizer) and similarity computation (cosine similarity).

8. Movie Dataset (TMDB 5000 Movies & Credits)

Kaggle. (n.d.). TMDB 5000 Movie Dataset. Retrieved from <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>

Contains metadata including genres, cast, crew, overview, and more.